

Research Report

Toy Blockchain and Ledger Simulator

All experiments below were run against the actual `toychain` binary and `internal/chain/internal/block` code in this repository, not against a description of it. Commands are given so the results can be reproduced.

1. Tamper-evidence

Setup. A fresh chain (`-difficulty 3`, to keep mining near-instant for the demo) was funded and mined into two blocks:

```
go run ./cmd/toychain faucet -data /tmp/tamper_demo.json -difficulty 3 -to Alice -amount 100
go run ./cmd/toychain mine -data /tmp/tamper_demo.json # -> block 1: FAUCET -> Alice : 100
go run ./cmd/toychain addtx -data /tmp/tamper_demo.json -from Alice -to Bob -amount 30
go run ./cmd/toychain mine -data /tmp/tamper_demo.json # -> block 2: Alice -> Bob : 30
```

Before tampering:

```
$ go run ./cmd/toychain validate -data /tmp/tamper_demo.json
VALID: chain integrity confirmed
```

The tamper. The data file was edited directly (not through the CLI, which would refuse an invalid transaction) to change block 1's transaction amount from 100 to 100000, leaving every other field — including the block's stored `hash` — untouched:

```
before: [{ 'sender': 'FAUCET', 'recipient': 'Alice', 'amount': 100 }]
after:  [{ 'sender': 'FAUCET', 'recipient': 'Alice', 'amount': 100000 }]
```

After tampering:

```
$ go run ./cmd/toychain validate -data /tmp/tamper_demo.json
INVALID: block 1: stored hash does not match recomputed hash
```

```
$ go run ./cmd/toychain balances -data /tmp/tamper_demo.json
Alice: 99970
Bob: 30
```

Why this happens. A block's hash is computed by `ComputeHash` over `{Index, Timestamp, Transactions, PrevHash, Nonce}` (`internal/block/block.go`). `Transactions` is part of that input. Editing the amount inside a transaction changes the bytes fed to SHA-256, so the **correct** hash of block 1 is now different from the hash value still sitting in the JSON file (which was computed before the edit and was never recalculated). `Validate` walks the chain from genesis and, for every block, compares the stored hash against a fresh `ComputeHash()` call (`internal/chain/chain.go`, first check inside the loop). That comparison is the exact check that fails here, and it fails at block 1 - the first and only block touched - which is what `Validate` reports (`FailedIndex`).

Notably, `balances` (which just replays transactions without checking hashes) **does** reflect the forged 100000, because nothing in `Ledger()` looks at hash integrity - it trusts whatever transactions are physically present in the file. That is precisely why a **separate**, explicit `validate` step matters: a system that only ever computes balances and never re-verifies hashes would silently accept this forgery. Tamper-evidence comes from the hash check, not from the balance computation.

One consequence worth calling out: because block 2's `PrevHash` is a copy of block 1's **stored** (now-stale) hash rather than a recomputation, the `PrevHash` link check between block 1 and block 2 still technically "passes" - it was the block-1 stored-vs-recomputed hash check that caught the tamper, one step earlier in `Validate`'s per-block loop than the link check. If the attacker had also patched block 1's stored hash to match the tampered content, the stored/recomputed check on block 1 would pass, but the link check on block 2 (`b.PrevHash != prev.Hash`) would then fail instead, because block 2 still carries the **original** hash of block 1 in its `PrevHash` field. Either way, altering an old block without re-mining everything after it is caught - just by a different one of `Validate`'s checks depending on how much the attacker edits.

2. Difficulty versus effort

Setup. A small standalone harness mined the same fixed block (one transaction, fixed timestamp, genesis as previous hash) at difficulties 1 through 6, three times each, using `internal/block's Mine` directly (no CLI/JSON overhead). Difficulty is "required leading hex-zero digits," as implemented in `block.MeetsDifficulty`.

Difficulty (hex zeros)	Attempts (nonces tried)	Avg. duration	Implied hash rate
1	15	0.10 ms	~150k/s*
2	403	0.24 ms	~1.67M/s*

3	5,510	2.99 ms	~1.85M/s
4	11,495	6.02 ms	~1.91M/s
5	419,085	203.7 ms	~2.06M/s
6	61,803,008	29.5 s	~2.10M/s

* At difficulties 1–2 the run finishes in under a millisecond, so the implied hash rate is dominated by timer resolution noise, not a real measurement; hash rate stabilizes around ~2M hashes/sec once the run is long enough to average out that noise (difficulty ≥ 3).

Trend. Attempts do not grow linearly with difficulty -they grow roughly geometrically, consistent with the underlying probability model. Each hex digit of the hash is (as a first approximation, since SHA-256 output is uniformly distributed) equally likely to be any of 16 values, so the chance a random hash has **at least** d leading zero digits is $1/16^d$. Finding a satisfying nonce is then a sequence of independent Bernoulli trials with success probability $p = 1/16^d$, and the number of attempts until the first success follows a geometric distribution with **expected value** $1/p = 16^d$. Going from difficulty d to $d+1$ multiplies the expected search space by 16 - so effort is exponential in difficulty, not linear or even polynomial.

The observed attempt counts bounce around that expectation rather than matching it exactly (e.g. difficulty 4's 11,495 actual attempts undershoots its 65,536 expectation, while difficulty 6's 61.8M overshoots its 16.78M expectation by ~3.7x). That's expected: a geometric distribution has a standard deviation on the same order as its mean, so a **single** trial at each difficulty is a noisy estimate of the expected effort - exactly why the report tests multiple difficulties rather than reading too much into one data point. What doesn't move despite that noise is the **rate of increase** between difficulty levels - roughly 15–150x per +1 step here, squarely in the same exponential regime as the 16x theoretical prediction, and utterly unlike the ~1x-per-step growth linear effort would produce.

This is also why the CLI's default difficulty is 4: at ~6 ms and ~11.5k hashes, mining is imperceptible, while difficulty 6 already costs half a minute on this machine - the non-functional requirement that "the reviewer should not wait minutes" rules out defaulting any higher than about 5 on typical laptop hardware.

3. Design write-up

Hashing scheme. `internal/block/block.go` defines an unexported `hashPayload` struct with exactly five fields, in this order: `Index`, `Timestamp`, `Transactions`, `PrevHash`, `Nonce`. `ComputeHash`

copies a block's values into a `hashPayload`, JSON-marshals it, and SHA-256-hashes the resulting bytes. Two design choices matter here:

- **`Hash` is excluded from its own input.** It's the output of hashing,

not a parameter to it; including it would make the hash unable to authenticate anything (you could set `Hash` to whatever you like and it would trivially "match itself").

- **Field order is fixed by the `hashPayload` type, and `encoding/json`**

marshals Go struct fields in declaration order (map keys would **not** have this guarantee, which is why account balances - a map - never feed a hash directly, only the ordered `[]Transaction` slice does). That gives `ComputeHash` the determinism required by FR-3: identical field values always serialize to identical bytes and therefore hash identically, on any run, on any machine.

Validation guarantees. `Chain.Validate` (`internal/chain/chain.go`) establishes integrity across the **whole** chain, not just one block, through a loop where each iteration depends on the previous block's already-verified hash:

1. **Genesis is checked against its own fixed invariants** (index 0, the

well-known `PrevHash`, and its hash matching a recomputation) rather than a proof-of-work target, since it's a constant, not mined data.

2. **For every later block:** its stored hash must equal a fresh

recomputation (catches any single-block tamper, as in Section 1); its `PrevHash` must equal the **actual previous block's** hash (catches splicing in a substitute block, or leaving a stale link after an edit); its hash must meet the chain's proof-of-work difficulty (catches a block that was never actually mined); and its index/timestamp must be consistent with the block before it (catches reordering or invented heights).

3. **Validation stops at the first failure** and reports which block and

which check failed (`Result.FailedIndex`, `Result.Reason`), rather than just a boolean, so a caller knows exactly where to look.

Because each block's `PrevHash` is **part of the next block's hash input**, tampering with block `k` - even if the attacker also patches block `k`'s stored hash to hide it - changes a value that block `k+1`'s stored hash was computed over. To make the whole chain "consistent" again, an attacker would have to re-mine block `k+1` (find a new nonce satisfying the difficulty target for the new content), which changes block `k+1`'s hash, which is embedded in block `k+2`, requiring it to be re-mined too, and so on to the tip. `Validate` therefore doesn't just check individual blocks in isolation - the chained hash inputs mean that leaving **any** later block unmodified after an early edit is what actually gets caught.

4. Discussion questions

How does the previous-hash link make tampering with an old block impractical in a real chain, even though it's trivial in this local toy? In this toy, tampering is trivial because the "attacker" has direct write access to the one JSON file that is simultaneously the only copy of the chain and the only source of truth - there's nobody to disagree with. In a real network, many independent nodes each hold and independently verify their own copy of the chain, and blocks are only accepted after they satisfy the proof-of-work (or equivalent) rule **and** match what peers already have. The previous-hash link is what makes an edit to an old block expensive there rather than merely detectable: as shown in Section 3, changing block *k* forces re-mining *k* and every block after it to keep the chain internally consistent up to the tip, and in a real network "the tip" is moving forward as honest miners keep extending the **original** chain. An attacker isn't just redoing some hashing; they're racing the combined hash power of the entire honest network to re-mine a whole suffix of blocks before it gets any longer - the "51% attack" cost. Locally, there's no race to lose and no peers to convince, so the same link that would be economically prohibitive to break in public is only **informationally** prohibitive here: `validate` will tell you it's broken, but nothing stops you from editing the file.

Proof-of-work is one way to decide who may add the next block. Name at least one alternative and give one advantage and one drawback versus proof-of-work. Proof-of-stake is the standard alternative: instead of spending computation to earn the right to propose the next block, validators put up an economic stake (their own coins) that can be destroyed ("slashed") if they act dishonestly, and the right to propose is allocated roughly in proportion to stake. **Advantage**: it doesn't require burning real-world energy on hashing that has no purpose beyond the lottery itself, so it's dramatically cheaper to run at scale (this is the reason Ethereum moved from PoW to PoS). **Drawback**: it makes influence proportional to existing wealth in the system ("the rich get richer" / plutocracy concerns), and its security argument is more circular than PoW's — it relies on the value of the stake being worth protecting, whereas PoW's cost is anchored in external resources (electricity, hardware) independent of the token's own price.

List three concrete ways this toy differs from a production blockchain. Pick one and sketch how you'd add it.

4. **No consensus among peers.** There's exactly one copy of the chain, so

"the longest valid chain" and fork resolution never come up - the file is authoritative by fiat, not by agreement.

5. **No transaction signatures.** Any caller can submit `-from Alice` with

no proof they control Alice's funds; the ledger only checks the **amount** is affordable, never **authorization**.

6. No Merkle tree. A block's transactions are hashed as a flat ordered

list directly inside the block hash; a production chain instead hashes transactions into a Merkle tree and only the root goes into the block header, enabling compact proofs that a given transaction is included without needing the whole block (used by light clients).

Sketch: adding digital signatures. Give each account a public/private key pair (crypto/ed25519 in the standard library is a natural fit - no external dependency needed). Extend ledger.Transaction with a PubKey and Signature field. When a transaction is created, the sender signs a canonical encoding of {Sender, Recipient, Amount} with their private key; ledger.Transaction.Validate would then also verify ed25519.Verify(PubKey, message, Signature) and confirm the PubKey actually hashes/derives to the claimed Sender identity (e.g., make Sender *be* a fingerprint of PubKey, so there's no separate identity to spoof). Rejecting an unverifiable signature the same way Validate currently rejects a non-positive amount would close the "anyone can spend anyone's funds" gap - the single largest gap between this toy and something that could actually hold value.

Sources

- SHA-256 and general hash function properties: Go standard library docs for crypto/sha256 (<https://pkg.go.dev/crypto/sha256>).
- Ethereum's proof-of-stake transition ("the Merge") as the PoS example referenced above: <https://ethereum.org/en/roadmap/merge/>.
- Ed25519 as the suggested signature scheme: Go standard library docs for crypto/ed25519 (<https://pkg.go.dev/crypto/ed25519>).